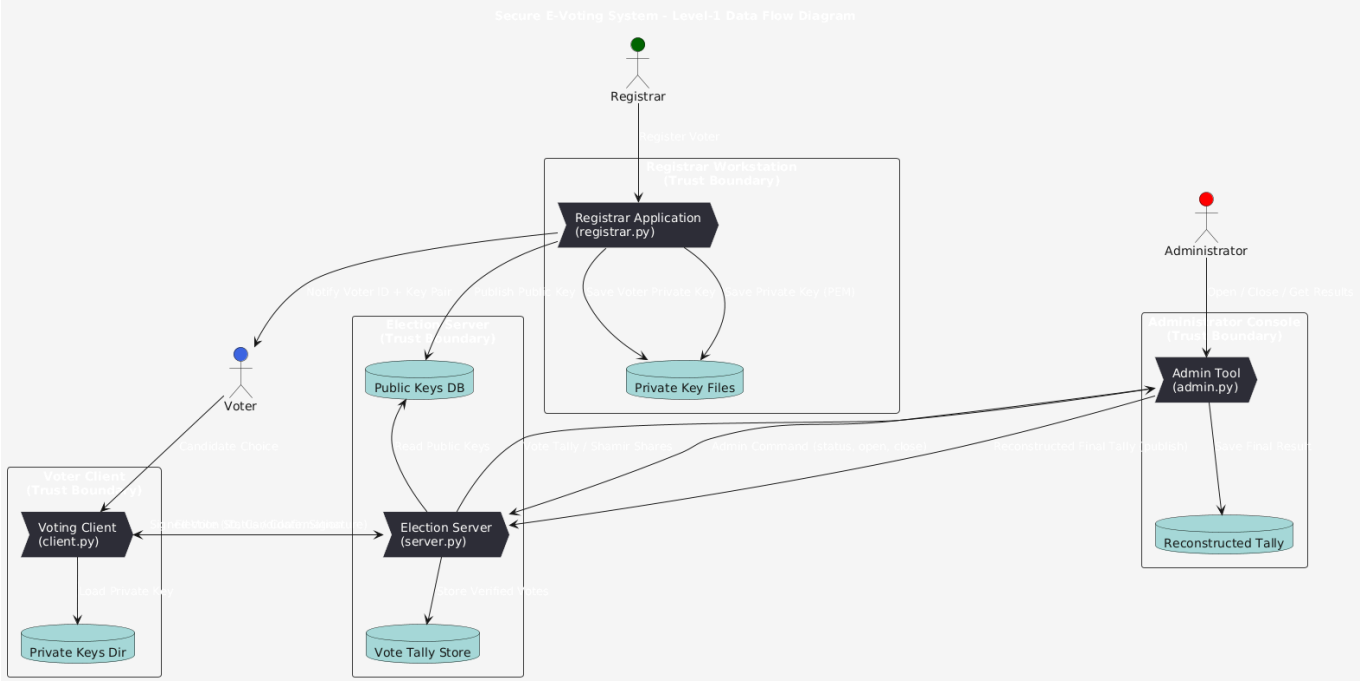

Part 1: Code Explanation & System Architecture

System Overview

This e-voting system consists of five main components: a Registrar, a Voting Client, a Voting Server, an Administrator, and a Shamir Secret Sharing module. It utilises Elliptic Curve Digital Signature Algorithm (ECDSA) for voter authentication and vote integrity, and Shamir Secret Sharing for securing the tally reconstruction process, ensuring that no single entity can tamper with or unilaterally reveal election results.

Component Roles

The system comprises five main components: the Registrar (ECDSA key generation), Voting Client (signs/submits ballots), Server (verifies votes and tallies results), Administrator (manages election state and tally reconstruction), and Shamir Module (handles secret sharing and reconstruction).

Interactions

1. Registrar generates keys → Server loads public keys.
2. Admin opens election → Voters submit signed votes → Server verifies and counts.
3. Admin closes election → Server generates Shamir shares → Admin reconstructs tally → Server publishes results.

Cryptography:

1. ECDSA is used to ensure vote integrity.
2. Shamir's Secret Sharing is used to prevent single-point tally exposure.

Configuration:

Main settings (server URL, share threshold, directories) are adjustable in each script.

End-to-End Test Suite and Instructions

The *test_script.sh* file simulates the entire election process from registration to result viewing on execution.

Part 2: Analysis of the Solution

Introduction:

This system was designed to provide an end-to-end secure electronic voting workflow for a small controlled environment. The main security goals were: (1) authenticate voters and ensure vote integrity, (2) protect the confidentiality and integrity of the final tally, (3) provide an auditable trail of ballots, and (4) limit trust in any single party. The following design decisions implement these goals while balancing performance, usability and complexity.

Summary table:

Design decision	Security problem addressed	Primary trade-offs
Use ECDSA (ECDSA/SHA-256) for voter signing (*client.py*, *registrar.py*, *server.py*)	Authentication of voters; integrity of submitted ballots	Requires key management (private keys on client devices); relies on secure private key storage; signing/verification costs (minimal)
Shamir Secret Sharing for final tally (*shamir.py*, *server.py*, *admin.py*)	Prevent single-party premature disclosure/manipulation of final tally	Adds complexity for closure/reconstruction; requires key share distribution & secure storage
Client-Server architecture over HTTP local testing (*client.py* ↔ *server.py*)	Centralised vote collection and signature verification	Central point of failure/trust; in production would require TLS and

		hardened server
Server-side public key DB (*public_keys.json*)	Central reference for verifying voter signatures	Single source of truth; risk if DB corrupted, needs integrity protection & access controls
Minimal audit trail: store ballots in memory (BALLOTS)	Enables basic auditing of received ballots	Memory storage is volatile (not persistent); needs tamper-evident logging for robust auditability

Design rationale and trade-offs

ECDSA signatures for voter authentication and integrity

Method: The *registrar.py* module is built responsible for registering voters by generating and managing cryptographic key pairs. For each voter, it creates an ECDSA private key (stored locally for the voter) and the corresponding public key (published to a central database accessible by the Voting Server). This means that each vote is signed on the client-side with ECDSA private key and the server verifies it using the corresponding public key.

Purpose:

- Ensuring only registered voters can submit valid votes. (authentication)
- Protects each ballot's integrity and non-repudiation. (a valid signature implies the holder of that private key submitted that ballot).

Trade-offs & limitations:

- **Key management burden:** Voters' private PEM files must be stored securely. If a private key were to be leaked, an attacker could use it to forge votes. OTPs or hardware tokens would reduce this risk but increase complexity and costs.
- **Usability vs security:** Storing unencrypted private PEMs on disk is convenient in secure/isolated environments however very risky in practice, using encrypted keys (password-protected) or platform keystores (e.g., OS keychain, smartcard) increases complexity but improves security.
- **Replay / timing:** The current design signs the static message *voter_id,candidate*. Without timestamps/nonces the system is vulnerable to replay if signature reuse is possible in other contexts; adding nonces or server challenge–response would reduce risk.

Shamir Secret Sharing (SSS) to protect the tally

Method: When election closes, the "*shamir.py*" module is introduced responsible for splitting each candidate's tally into multiple (*N*) and reconstructing it from a sufficient subset of those shares also known as the threshold (*T*).

Purpose: Multiple administrators/operators are required to collaborate in order to meet the threshold and reconstruct the tally. This defends against insider threats and single-point manipulation of results. This means ideally a single admin should not be able to reconstruct the tally on their own, unless they hold all the shares to begin with.

Trade-offs & limitations:

- **Availability:** If too many shares are lost or too many admins are unavailable, the tally may be irrecoverable. Setting *T* balances security with required availability for reconstruction.
- **Operational complexity:** Admins must coordinate to retrieve shares and perform reconstruction. Mistakes in share handling can prevent reconstruction.

Centralised server for vote collection and validation

Method: The *server.py* is the central component that manages the election state, accepts and validates signed votes (verifying signatures against the "*public_keys.json*" database, and increments *VOTE_COUNTS* accordingly. Once the election is "CLOSED" the server does not directly reveal the final tally, but rather distributes shares of it.

Purpose: Central verification simplifies state management and can prevent multiple votes per person if configured so. (This current implementation allows for multiple votes per voter for testing/demonstration purposes).

Trade-offs & limitations:

- **Single point of failure:** The server has to be trusted to not fail or to not corrupt tallies prior to share generation.
- **Scalability and DoS:** A single server can be overwhelmed. Rate limiting would be considered in production.
- **Network security:** The current implementation uses HTTP for local testing; which is not very secure against eavesdropping and man-in-the-middle attacks.

Public key database

Method: Registrar aggregates voter public keys into a simple JSON file (*public_keys.json*) accessed by the server.

Purpose: Provides an authoritative mapping between voter IDs and their corresponding public keys, enabling the server to authenticate voters and prevent impersonation or unauthorised vote submissions.

Trade-offs & limitations:

- **Integrity:** A single JSON file can be corrupted or tampered with.
- **Revocation / updates:** The system currently assumes keys are static. Key revocation or rotation requires an admin workflow and authenticated key updates. (*reload_keys()*)

STRIDE Threat Model

Spoofing Identity (S)

Threat: Voter impersonation
Scenario: An attacker submits votes using another voter's ID and signature.
Affected components: *client.py* (signing), *server.py* (verification), *public_keys.json*.
Mitigation:

- **Enforce private key protection:** Saving the keys with a password or storing them in a secure place managed by the operating system (like a keychain or credential vault).
- **Add server nonce/timestamp before signing:** This way every signed message is unique and prevents "replay" of old signatures.

Explanation: Protecting the private keys reduces the chance of forged signatures meaning lower risk for unauthorised votes being accepted. Server nonce/timestamps prevent replay attacks by adding small random code that ensures each signed message is unique and cannot be replayed/reused.

Threat: Administrator/registrar impersonation (unauthorized admin commands)
Scenario: An attacker sends "*/open*", "*/close*", "*/reload_keys*" or any other admin commands to which they should normally not have access to.
Affected components: *admin.py* and *server.py*.
Mitigation: Require admin authentication (e.g., mutual TLS, API tokens tied to admin accounts). Additionally logging all admin actions can help identify and traceback any malicious commands.
Explanation: Authentication can help prevent unauthorized control; logs provide traceability.

Tampering (T)

Threat: Ballot tampering in transit
Scenario: An attacker modifies the submitted vote (*voter_id* or *candidate*) in transit to the server.
Affected components: *client.py*, *server.py*.

Mitigation: Use TLS (HTTPS for the web pages) for all communication between the client and the server and verify the ECDSA signature on the server for each ballot.
Explanation: TLS prevents in-transit modification and the signature verification step can still detect tampering even if TLS fails.

Threat: Tampering with *public_keys.json*
Scenario: An attacker or malicious insider modifies public keys or vote counts.
Affected components: *public_keys.json*, *server.py*.
Mitigation: Sign *public_keys.json* by the registrar and verify the signature on load.
Explanation: Signing/verification ensures authenticity of the key DB.

Repudiation (R)

Threat: Voter or server repudiation (dispute about whether a vote was cast/accepted)
Scenario: A voter claims they did not vote and server logs are missing or ambiguous.
Affected components: *client.py*, *server.py*, *BALLOTS*.
Mitigation: Maintain signed audit logs (ballot receipts) returned to voters (e.g., a signed receipt containing vote hash and timestamp).
Explanation: Signed receipts and audit logs create non-repudiable evidence of actions.

Information Disclosure / Confidentiality (I)

Threat: Leak of private keys (attacker gains ability to forge votes)
Scenario: An attacker obtains voters' private PEM files from *private_keys* directory.
Affected components: *client.py*, *private_keys* DB.
Mitigation: Encrypt private keys with passphrases and/or use OS-provided key stores. Encourage offline storage of private keys (USB token).
Explanation: Encrypted or hardware-protected keys have a drastically lower chance of becoming compromised.

Threat: Disclosure of raw ballots linking voter to candidate
Scenario: Server logs or *BALLOTS* can reveal voter to candidate mapping.
Affected components: *server.py*, *BALLOTS*, logs.
Mitigation: Minimize personally-identifying information in logs, store only necessary metadata, rotate or redact logs, and encrypt logs at rest.
Explanation: Minimizing stored personal data and encryption reduces privacy risk and data exposure in breaches.

Denial of Service (D)

Threat: DoS against the voting server (attempt to prevent voting or tallying)
Scenario: High-rate malicious traffic overwhelms the server, preventing legitimate votes from being processed in time.

Affected components: *server.py*, HTTP *network*.
Mitigation: Rate limiting, request authentication, resource limits, and deployment of other special DDoS prevention tools in the background. Maintain offline logging and request queueing.
Explanation: the utilisation of such anti-DoS methods reduce the impact of volumetric attacks.

Threat: Loss of shares or admins unavailable (inability to reconstruct the tally)
Scenario: Some share files are lost or administrators cannot cooperate, making reconstruction impossible.
Affected components: *TALLY_SHARES*, tally reconstruction/publication and the other admin processes.
Mitigation: Choose threshold T conservatively (trade-off with confidentiality) and implement recovery procedures and documented SLAs for admins.
Explanation: Documented recovery procedures and conservative thresholds improve availability and provide backup solutions for incidents. This can sometimes however impact secrecy.

Elevation of Privilege (E)

Threat: Unauthorized privilege escalation on server or registrar host
Scenario: Exploited vulnerability in server or OS allows an attacker to change code or DB.
Affected components: Server host, registrar host, *public_keys.json*, share storage.
Mitigation: Run services in containers or restricted user accounts. Implement role separation and multi-factor admin authorisation.
Explanation: Isolated/contained services operated in restricted user accounts reduce attack surface and limit what any compromise can do significantly.

Threat: Admin account compromise leading to publication or manipulation of tally
Scenario: Attacker takes over an admin account and publishes a forged tally or reconstructs using stolen shares.
Affected components: *admin.py*, admin processes, server publish endpoint.
Mitigation: Enforce extra admin authentication such as MFA, potentially require multi-party approval before major changes/publishing (e.g., two admin signatures), and log signed tally receipts to allow verification.
Explanation: Strong auth and multi-party controls prevent a single compromised admin from acting alone.

Attackability assessment

Attack	Description & impact	Existing control	Additional mitigation
Private-key theft	Attacker steals	ECDSA	Encrypt keys or

	voter PEM file and votes fraudulently	verification ensures only key holder can sign, but assumes local file security	store in OS keychain
Replay of signed votes	Old signed messages reused	Server checks signature validity but not timestamps	Include server nonce
Server manipulation of tally	Server could modify counts before share split	Shamir sharing + logs give partial protection	Use verifiable secret sharing, signed logs
DoS on server	Flooding prevents voting	Simple in-memory rate limiting mechanism	Rate-limit requests; deploy behind proxy

Secure design methodology evaluation

Least Privilege:
Each process runs only the tasks necessary for its role: the registrar handles key generation, the client signs votes, and the server verifies and tallies. No component holds both registration and tally control. However, in the current implementation all services run under the same user account, so a compromise could escalate privileges. Running components in isolation or separate OS users with controlled privileges would enforce the principle more effectively.

Separation of Privilege:
The system conceptually separates authority over different security domains: the registrar manages identities, while administrators reconstruct tallies using Shamir shares. This separation limits single-point abuse. In practice, because the admin can collect all shares locally, the privilege separation is simulated rather than enforced. Distributing shares to independent custodians would make it genuine.

Part 3: Case Study

Introduction

This case study examines the Sybil attack on BitTorrent's Distributed Hash Table (DHT). BitTorrent's DHT is a decentralised lookup service that enables peers to locate content without a central tracker. Its open, permissionless nature means any peer can join and choose its own identity making it efficient however also susceptible to identity-based attacks. Sybil attack occurs when a single adversary creates a large number of fake identities to control routing and data storage. This analysis

outlines the DHT's architecture, details the Sybil attack, reviews the main mitigations adopted by BitTorrent, and reflects on how secure design principles could have prevented the weakness.

System architecture

BitTorrent's Distributed Hash Table (DHT) is based on the Kademlia routing algorithm (Maymounkov & Mazieres, 2002; BitTorrent Enhancement Proposal 5, 2008), which remains the foundation of most large-scale peer-to-peer lookup systems (Urdaneta et al., 2011; Zhang et al., 2020).

Each peer generates a unique 160-bit Node ID and maintains "k-buckets" (lists of contacts) grouped by their distance in the identifier space. File metadata (an info-hash) is stored and located using Kademlia's XOR distance metric, which routes queries through nodes numerically closest to the target key.

Because peers self-assign IDs and join via bootstrap nodes, there is no central authority to verify identity authenticity. This openness enables scalability but also allows a single attacker to create many fake nodes, forming the basis of a Sybil attack.

Technical details of the attack

A Sybil attack on a DHT is performed when an adversary creates many logical identities (nodes) under its control, so that lookups and routing in the overlay are dominated by attacker-controlled peers (sybils). In a Kademlia-style DHT like BitTorrent's Mainline DHT, this undermines the routing substrate: honest nodes route queries toward the keys using nodes that are numerically "close" in the ID space, so controlling many IDs gives the adversary an outsized role in answering queries for targeted info-hashes (Douceur, 2002; Crosby & Wallach, 2007).

Step-by-step attack sequence:

1. Mass identity generation: the attacker boots many DHT nodes (thousands, if possible) and assigns each a Node ID. IDs can be random or intentionally selected to be near target info-hashes to maximize control of relevant routing buckets (Crosby & Wallach, 2007).

2. Bootstrap & flooding: the attacker uses bootstrap contacts and aggressive probing (ping/announce) to advertise its Sybil nodes so that honest peers add them into their k-buckets. Because k-buckets favor recently seen or reachable peers, active Sybils can gradually replace legitimate entries.

3. Routing table pollution: over time the attacker's nodes occupy the closest entries for many keys in many honest peers' routing tables. This is often

accelerated during churn (node restarts) when honest nodes refresh their contacts from attacker-controlled addresses.

4. Lookup interception & manipulation: when victims perform lookups for a target info-hash, the iterative routing will route queries through attacker nodes. The attacker can then (a) return no peers (causing lookup failure/DoS), (b) return attacker-controlled peers that serve malicious or poisoned content, or (c) log/monitor queries to infer which torrents a victim is interested in (privacy breach) (Urdaneta et al., 2011).

5. Persistence & amplification: to sustain control despite churn or countermeasures, attackers may use many IPs (botnets/cloud VMs) or exploit relaxed Node ID checks to rejoin quickly.

Response to the attack

BitTorrent's DHT introduced several defences to reduce the impact of Sybil attacks by raising the cost of creating many fake identities.

Node-ID binding (BEP 42).
BitTorrent Enhancement Proposal 42 ties a node's ID to its IP address and port by hashing this pair, preventing one host from generating unlimited arbitrary IDs. This limits each network address to a single valid identity, increasing the resources required for a Sybil attack.

Subnet diversity and rate limiting.
Modern clients restrict how many routing-table entries can come from the same /24 subnet and prefer long-lived, responsive peers. They also limit the rate of new joins and blacklist nodes that behave abnormally or flood announcements. These rules keep routing tables geographically and topologically diverse, reducing the chance that Sybils dominate a victim's k-buckets.

Bootstrap and fallback safeguards.
Clients retain trusted bootstrap peers and refresh routing tables if lookup success drops, allowing them to recover from possible Sybil pollution rather than continuing operation with compromised state.

Overall, these measures significantly increase the effort required to launch large-scale Sybil attacks while preserving the decentralised nature of BitTorrent's DHT.
(Citations: Douceur 2002; Urdaneta et al. 2011; BEP 42 2013.)

Reflection

The Sybil attack on BitTorrent's DHT illustrates how unrestricted identity creation undermines trust in decentralised systems. Applying secure design principles could have reduced this risk.

The principle of least privilege suggests limiting new or unverified nodes' routing influence until they prove reliability, preventing any single entity from dominating lookups.

Defence in depth supports combining multiple safeguards such as ID binding (BEP 42), subnet diversity, and reputation checks so that the failure of one control does not compromise the whole network.

Stronger identity assurance (e.g., proof-of-work or lightweight certification) could further raise the cost of large-scale Sybil attacks while preserving decentralisation.

Bibliography

- BEP 5 (2008) BitTorrent Enhancement Proposal 5: DHT Protocol. Available at: https://www.bittorrent.org/beps/bep_0005.html (Accessed: 08 Nov 2025).
- BEP 42 (2013) BitTorrent Enhancement Proposal 42: DHT Security Extension. Available at: https://www.bittorrent.org/beps/bep_0042.html (Accessed: 08 Nov 2025).
- Crosby, S.A. & Wallach, D.S. (2007) 'An Analysis of the Sybil Attack on Peer-to-Peer Networks', IPTPS 2007.
- Douceur, J.R. (2002) 'The Sybil Attack', Proceedings of the 1st International Workshop on Peer-to-Peer Systems (IPTPS), 2002.
- Maymounkov, P. & Mazieres, D. (2002) 'Kademlia: A Peer-to-Peer Information System Based on the XOR Metric', IPTPS 2002.
- Urdaneta, G., Pierre, G. & van Steen, M. (2011) 'A Survey of DHT Security Techniques', ACM Computing Surveys, 43(2), Article 8.